Fast, flexible ordering for any business

Software Requirements Specification

Version 1

Team Number: 9

Project Manager: Kim Mayo

Mentor: Shibu Tewar

Team Members: Laura Alarcon, Sasha Cabrera, Simon Johnson-Bush, Kevin Navolanic, Owen Roberts

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Group 9	Initial version	10/12/2025
1.2	Group 9	Final Version	10/19/2025
2.0	Group 9	Initial Version 2	10/25/2025
2.1	Group 9	Final Version 2	11/2/2025

Review History

Reviewer	Version Reviewed	Date
Mr. Tewar	Version 2	10/26/2025

Table of Contents (Version 1)

4. High-level Design
    4.1. Security (Required)
    4.2. Hardware (Required)
    4.3. User Experience (Required)
    4.4. Architecture (Required)
    4.5. Database (Required)
    4.6. Top-level Classes (Required)
    4.7. Data Flow and States (Required)
    4.8. Reports (Required)
    4.9. Internal Interfaces (Optional)
    4.10. External Interfaces (Optional)
    4.11. Other Outputs (Optional)
    4.12. Configuration Data (Optional)
    4.13. Training (Optional)
5. Appendixes (Optional)
    5.1.

1. Introduction

1.1 Project Objectives

The goal of *Orderly* is to provide a flexible, user-friendly self-service ordering system that helps businesses efficiently manage customer orders without the need for costly, proprietary solutions. Many small and medium-sized businesses struggle with outdated or overly complex systems that require specialized training. *Orderly* solves this problem by offering an intuitive and scalable platform that can be easily adapted for restaurants, retail shops, and service providers.

The project aims to deliver:

- A working prototype of a customizable self-service ordering system.
- A simple interface for customers to browse items and place orders.
- Administrative tools for businesses to easily update products, menus, and prices.
- Improved operational efficiency, reduced wait times, and minimized human error.

1.2 Project Scope

In-Scope Features:

- Customer-facing interface for browsing, customizing, and submitting orders.
- Business-facing interface for managing menus, inventory, and prices.
- Order management system that tracks submissions through fulfillment.
- Basic reporting dashboard for sales summaries and popular items.
- Responsive design compatible with tablets, kiosks, and mobile devices.

Out-of-Scope (for this phase):

- Integration with payment gateways (prototype will simulate payments).
- Integration with external POS systems.
- Advanced analytics or AI-based recommendations.
- Loyalty programs or promotional features.
- Multi-language and multi-currency support.

Future Enhancements:

Future versions may include secure payment processing, customer loyalty programs, and advanced data analytics to provide deeper business insights. These features will be considered in later development cycles as the system scales to production.

1.3 Project Overview

Orderly is designed primarily for small to mid-sized businesses seeking affordable, efficient, and customizable digital ordering solutions that reduce labor costs and improve customer satisfaction.

Project Timeline:

- Sprint 1 (9/15–10/5): Group kickoff and project selection
- Sprint 2 (10/6–10/19): Requirements specification (SRS development)
- Sprint 3 (10/20–11/2): High-level system design
- Sprint 4 (11/3–11/23): Final presentation and prototype demonstration

Deliverables:
Weekly status reports, project overview, SRS versions, and a final presentation.

Success Criteria:
Project success will be measured by the delivery of a functional, user-friendly prototype that meets user requirements, demonstrates efficient order processing, and provides a scalable foundation for future system enhancements.

2. Project Description

2.1. Project Features / Functions

2.1.1. Inventory and Data Management
2.1.2. Order Customization and Order History
2.1.3. User Interface for tablets, kiosks, and mobile devices
2.1.4. Sales Dashboard for Weekly and Monthly

2.2. User Stories

2.2.1. As a business owner, I want to be able to quickly count and edit inventory, so my customers always know what is available.
2.2.2. As a business owner, I want to see my most popular items so that I can prioritize ordering decisions.
2.2.3. As a customer, I want to be able to search for items and customize my order.

2.3. Use Case

2.3.1.

Use Case ID	**UC1**
Use Case Name	Manage Inventory
Actor(s)	Business Admin
Precondition	Admin is logged into management software
Main Flow	1. Opens inventory and makes count of what they have for the day 2. Admin can add, edit, or remove items 3. Admin confirms inventory for the day 4. Customers can then order until an item hits 0, which will automatically inform admin they need to order more
Alternate Flow	Inventory at 0, would show automatically sold out
Postcondition	Menu is updated in real time for both business and customer

Use Case ID	UC2
Use Case Name	Customize Items/Order
Actor(s)	Registered Customer
Precondition	User has an active account/guest account
Main Flow	1. User can start a new order or select old order 2. User can search for items 3. User can save this custom item for later 4. The system will update the price of the item based off additional things they add for what they admin has listed is “extra”
Alternate Flow	The system then confirms if item is possible as customized or will display error that needs to be fixed
Postcondition	Order/Item is saved and redirected back to main menu to select more or proceed to checkout.

2.3.2.

Use Case ID	**UC3**
Use Case Name	Best Sellers
Actor(s)	Business Owner/Admin

Precondition	Admin logs into sales management
Main Flow	1. Sales Management will detect the best sellers based on quantity sold 2. Admin can select which items to prioritize for next shipment 3. Admin can also use data to determine how long something might stay a best seller, and develop something similar to the item to keep things fresh
Alternate Flow	(Future flow) System can automatically add the two best sellers to the next shipment, then admin can confirm to order
Postcondition	System will get a new best seller either weekly or monthly based on what admin wants

2.4. Project Assumptions and Dependencies

The development of Orderly relies on a number of key assumptions and dependencies that shape the scope, design, and delivery of the system. These elements are essential to ensure successful collaboration, efficient progress, and alignment with project objectives.

Assumptions:

- All team members have consistent internet access and functioning devices capable of running development and collaboration tools.
- Each member has access to shared resources such as Jira, Microsoft Teams, and Outlook to support communication, sprint planning, and documentation.
- Stakeholders and the project mentor will provide timely feedback during sprint reviews to help guide improvements and ensure alignment with requirements.
- The system will operate in a simulated environment during development, meaning no real payment gateway or POS integration is required for the prototype.

- Test data used for the prototype will be fictional to maintain data privacy and comply with ethical standards.

Dependencies:

- Jira will be used for agile project management, including backlog organization, sprint tracking, and user story management.
- Microsoft Teams and Outlook will serve as the main communication and scheduling platforms for team coordination.
- Google Drive will store shared project documents, presentations, and SRS drafts, ensuring version control and accessibility for all members.
- The system prototype depends on the availability of basic development tools and frameworks agreed upon by the team (e.g., HTML, CSS, JavaScript).
- Future enhancements, such as payment gateway integration or POS connectivity, will depend on the availability and compatibility of external APIs.

2.5. Project Collaboration and Documentation

Effective collaboration and consistent documentation are essential to the success of Orderly. The project team uses a structured combination of tools and communication methods to support transparency, accountability, and efficiency throughout the development.

- Jira serves as the primary tool for task management and sprint tracking. It allows the team to organize the product backlog, assign tasks, and monitor progress using Scrum boards and burndown charts. Each member updates their assigned tasks regularly to reflect current progress.
- Microsoft Teams is used for daily communication and collaboration, providing a central space for meetings, chat discussions, and file sharing. Weekly check-ins are held to review sprint progress, address any blockers, and plan upcoming tasks.
- Outlook supports scheduling and deadline management by sending reminders and meeting invitations, ensuring everyone remains aligned with key milestones.
- Google Drive is used to store and maintain all project documentation, including the Software Requirements Specification (SRS), design documents, meeting notes, and final presentation files.

- All documents follow consistent formatting and naming conventions to ensure clarity and professionalism across project deliverables.

This collaborative structure enables the Orderly team to stay organized, maintain clear communication, and efficiently produce high-quality deliverables that meet the project's objectives and deadlines.

2.6. Project Management

The *Orderly* development team will utilize the Agile Scrum methodology to manage the project efficiently, encourage collaboration, and ensure flexibility throughout the development cycle. This approach allows the team to adapt quickly to feedback and deliver incremental improvements at the end of each sprint.

Methodology

Agile Scrum was chosen for its focus on iterative development, team communication, and customer feedback. The project will be divided into four sprints, each lasting approximately two to three weeks. At the end of each sprint, the team will review progress, demonstrate completed features, and identify areas for improvement before beginning the next cycle.

Sprint Planning

Each sprint will begin with a Sprint Planning Meeting, where tasks are selected from the product backlog and assigned priorities.

The team will break down user stories into smaller, actionable tasks. These tasks will be estimated for effort and assigned to individual team members based on skill set and workload balance.

Task Assignment and Progress Tracking

Tasks and sprint progress will be tracked using Jira, which allows for visual management through Kanban boards and sprint burndown charts. Each member will update their task status daily to reflect ongoing progress.

Microsoft Teams will be used for communication, file sharing, and virtual meetings, while Outlook will manage scheduling and deadlines. Weekly e-mails will ensure accountability and identify any blockers early.

Review and Retrospective

At the end of each sprint, the team will hold a Sprint Review to present completed features and gather feedback from stakeholders. A Sprint Retrospective will follow to discuss what worked well, what challenges were encountered, and how the process can be improved for the next iteration.

This structured yet flexible project management approach will help ensure that *Orderly* is delivered on time, meets all requirements, and maintains a high standard of quality throughout development.

3. Requirements Specification

3.1. Business Requirements

ID	Description	MOSCOW
BR1	Application must be available for download through the Google Play Store, Apple Store, as well as from Orderly's main website	Must
BR2	Application could link to payment processing service of choice for handling of online payments and deposits to businesses bank account	Could
BR3	The back-end application must provide analytics on ordering frequency from customers and track tendencies and trends for popularity of items sold	Must
BR4	Application must track inventory and count availability of menu items/stock vs amount of inventory available in real time.	Must
BR5	Ordering of supplies could be extended to other employees at the permission of the admin. I.E. assistant managers or shift leads	Could

ID	Description	MOSCOW
BR6	Suppliers and store could be able to communicate through app by having DM privileges for real time communication without relying solely on email	Could

3.2. User Requirements

ID	Description	MOSCOW
UR1	User must be able to create an account through a main page link from initial startup of the app	Must
UR2	User data must be stored securely and only accessible through a unique email and password set up by the user upon account creation.	Must
UR3	A unique profile for the user should track their order history and allow for reordering from previous suppliers without doing a new search every order period.	Should
UR4	Optional email signup for access to promotions and offers from participating suppliers could be offered.	Could
UR5	The system should provide smooth operating GUI for touch screen phones.	Should
UR6	Payment processing must be streamlined. User must be able to enter payment info and process their order from one screen and receive a payment confirmation immediately.	Must
UR7	The system could provide a rating system to provide feedback or ordering experience, delivery times, and general user experience.	Could

3.3. Functional Requirements

ID	Description	MOSCOW
FR1	The system must authenticate users through a secure login module.	Must
FR2	The system must allow users to browse a catalog of items (menu, products or services). The navigation should be intuitive.	Must
FR3	The system must allow customers to select and customize orders. The system must allow submission and show a confirmation. The system must update order status based on current data (Pending, In progress, Ready, Completed)	Must
FR4	The system should generate summaries of sales and volume to track trends. (Daily / Weekly total and top items)	Should
FR5	The system could have a dark mode for accessibility via toggle.	Could
FR6	The system must provide an interface for businesses to manage their products and services. This includes providing administrative access to authorized users.	Must
FR7	The system must log all orders with timestamps and customer details, as well as item list.	Must
FR8	The system must adapt to multiple devices and stay functional in portrait and landscape orientations.	Must

3.4. Non-Functional Requirements

ID	Description	MOSCOW
NFR1	User accounts must have proper security when handling private data (i.e. passwords, payment information, emails, etc).	Must

ID	Description	MOSCOW
NFR2	System must process tasks, like searches and ordering, quickly.	Must
NFR3	The system could be able to handle a decent influx of users, preferably over 10 million.	Could

3.5. Implementation (Performance) Requirements (Optional)

ID	Description	MOSCOW
IR1	e.g., The development environment shall use Visual Studio Code and GitHub.	M
IR2		
IR3		

4. High-level Design

4.1. Security (Required)

4.1.1. Specification 1

Data encryption will be the main asset of security features. Since users on both the business and buyer side will need their secure financial data and personal information uploaded to the app this system will need AES-256 to make sure the data at rest is encrypted and secure.

Resting data is all the information stored in the databases and servers of the app that would be considered sensitive. An encryption key will be needed and assigned at every user sign up so only they have access to their personal data even when they are logged off the app.

4.1.2. Specification 2

This app will also need role based authentication and permission. RBAC will separate roles between the business owners and the shoppers on the app. Business owners will need their own server side for their roles in the app and the consumers buying from the app will need their own server side.

Users should only be able to browse and purchase from the app and business should be able to adjust their inventory and menu items from their business page.

A user should not be able to change or alter menu composition and inventory count. Business should also not be able to access purchasing power from the shopping users.

4.2. Hardware (Required)

4.2.1. Specification 1

Requirement ID: HW1
Description: Network Connectivity
Requirement: The deployment environment should have a stable broadband internet connection to support real-time data synchronization between the customer interface, order management system, and administrative dashboard.

4.2.2. Specification 2

Requirement ID: HW2
Description: Data Storage
Requirement: The system should utilize at least 100 GB of secure storage, hosted locally or in the cloud, to maintain application data, order logs, and customer activity records.

4.2.3. Specification 3

Requirement ID: HW3
Description: Client Devices
Requirement: The system should operate on standard web-enabled devices, such as touchscreen tablets and kiosks for customer use and desktop or laptop computers for business management, each equipped with modern web browsers (e.g., Chrome, Edge, Safari).

4.3. User Experience (Required)

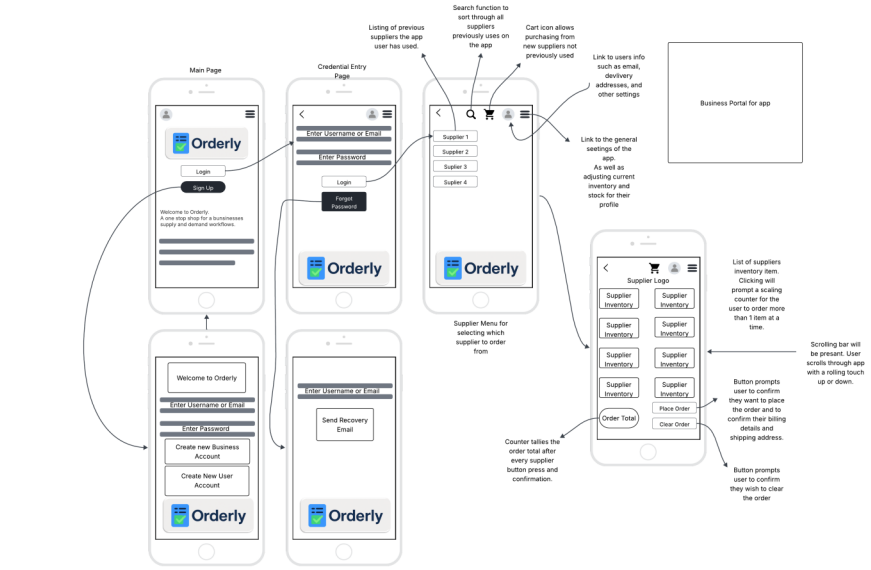
4.3.1. Specification 1

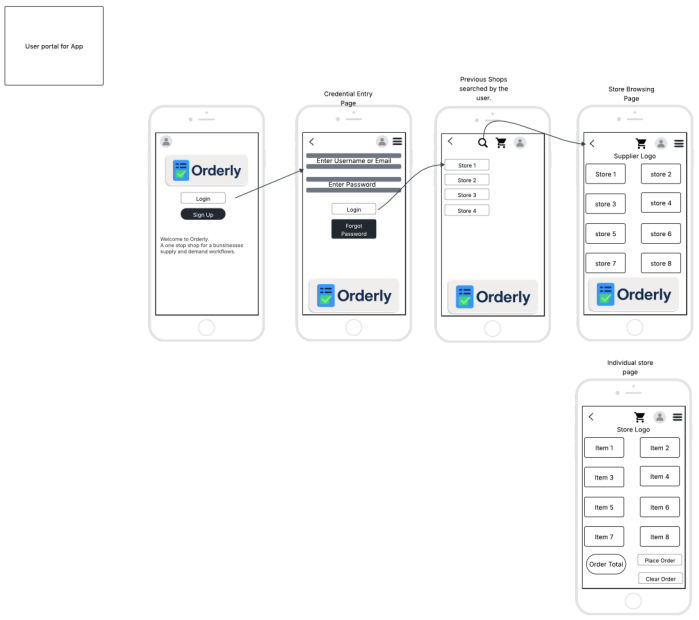
The interface will be split between a business account and a user account. Businesses will have their own pathways through the touch screen for updating their inventory counts, ordering more inventory, and interacting with the users.

4.3.2. Specification 2

With a mobile app design the touchscreen will provide buttons for the navigation from the login page with a username and password to the browsing page, to the ordering page, and finally the confirmation of order page.

4.3.3. User Interface Wireframes





4.4. Architecture (Required)

The architecture of Orderly defines the structure and organization of the software system, illustrating how the main components: frontend, backend, and database

work together to deliver a seamless experience for both customers and business administrators. The system will follow a client–Server Architecture, ensuring scalability, maintainability, and clear separation of user interface and business logic.

- **Requirement ID:** ARCH1
- **Description:** Client–Server Architecture
- **Requirement:** The system shall follow a client–server architecture where the frontend communicates securely with the backend through RESTful APIs over HTTPS. This structure will allow efficient communication between customers, businesses, and supplier management modules.

4.4.1. Specification 1

Frontend (Client Side):

- Developed using React.js or standard web technologies (HTML, CSS, JavaScript).
- Provides two primary user interfaces:
- Customer Interface: Allows customers to browse items, customize orders, and complete purchases.
- Business Interface: Enables business owners or administrators to manage inventory, update menus or product lists, and review sales data.
- Communicates with the backend through secure REST API requests for all user actions (login, browsing, ordering, and reporting).
- Designed to be responsive and adaptable to various devices such as mobile phones and tablets.
- Supports a simple, intuitive layout to improve accessibility and user satisfaction.

Backend (Server Side):

- Implemented using Node.js and Express.js frameworks.
- Handles all business logic, including user authentication, inventory tracking, order management, and data validation.
- Processes API requests from both customer and business interfaces, interacting with the MySQL database to fetch or update information.
- Supports modular API endpoints for future integration with external services such as payment gateways or supplier systems.

4.4.2. Specification 2

• All communication between the frontend and backend is encrypted using **HTTPS** to ensure data security.
• Authentication and authorization are managed through JSON Web Tokens (JWT) for secure user sessions.
• The backend validates and sanitizes data to prevent unauthorized access or injection attacks.
• The system is designed for deployment on cloud-based platforms such as AWS or Azure to enable scalability, reliability, and high availability.
• Future versions of the system may include load balancing and caching to optimize performance during high-traffic periods.

4.5. Database (Required)

Requirement ID: DB1

- **Description:** Order Management Database
- **Requirement:** The system shall use **MySQL** as the primary database management system to store and manage order-related data for restaurants, retail shops, and service providers. The database will support CRUD operations for customer information, order details, inventory, and payment tracking.

4.5.1 Specification 1

The database shall include the following entities and key attributes:

- Customer – CustomerID, FirstName, LastName, Email, Address, City, State, ZipCode, Phone
- Order – OrderID, , OrderItemID CustomerID, PaymentID, OrderDate, TotalOrder, Tax, TotalPaymentDue, Status
- OrderItem – OrderItemID, OrderID, ProductID, Quantity, UnitPriceCharged, ItemTotal
- Product – ProductID, SupplierID , ProductName, Category, UnitPrice, StockQuanity
- Payment – PaymentID, OrderID, PaymentType, Total, PaymentDate
- Supplier – SupplierID, SupplierName, SupplierAddress, SupplierCity, SupplierState, SupplierZipCode, SupplierPhone, SupplierContact, SupplierEmail
- SupplierOrders – SupplierOrderID, SupplierID, SupplierOrderItem, SupplierPaymentID, SupplierOrderDate, TotalSupplierOrder, Status
- SupplierOrderItem – SupplierOrderItemID, SupplierOrderID, SupplierID, ProductID, Quantity, UnitPricePaid, ItemTotalk
- SupplierPayments – SupplierPaymentID, SupplierOrderID, SupplierID, SupplierPaymentType, PaymentDate, TotalPayment, SupplierPaymentStatus

Each order is linked to one customer and may contain multiple order items. Products can appear in many order items, and each order may have one or more payments associated with it.

4.5.2. Specification 2

The system shall maintain **referential integrity** through the use of foreign keys:

- CustomerID in Order references Customer(CustomerID)
- OrderID in OrderItem references Order(OrderID)
- ProductID in OrderItem references Product(ProductID)
- OrderID in Payment references Order(OrderID)
- SupplierID in Product references Supplier(SupplierID)
- SupplierID in SupplierOrder references Supplier(SupplierID)
- SupplierOrderID in SupplierOrderItem references SupplierOrders(SupplierOrderID)
- SupplierOrderID in Payments references SupplierOrder(SupplierOrderID)

The database will also enforce constraints such as NOT NULL for required fields and UNIQUE for emails and ids.

4.5.3. Entity Relationship Diagram (ERD)

4.6. Top-level Classes (Required)

4.6.1 Specification 1

- **Requirement ID:** CLASS1
- **Description:** Customer Class
- **Requirement:** The Customer class shall store and manage customer information and provide methods for registration, updating contact details, and viewing order history.

  Attributes: CustomerID, FirstName, LastName, Email, Address, City, State, ZipCode, Phone

  Methods: register(), updateInfo(), viewOrderHistory()

4.6.2. Specification 2

- **Requirement ID:** CLASS2
- **Description:** Order Class

- **Requirement:** The Order class shall represent individual customer orders and manage their creation, modification, and completion.
- **Attributes:** Order – OrderID, , OrderItemID CustomerID, PaymentID, OrderDate, TotalOrder, Tax, TotalPaymentDue, Status
  Methods: addItem(), removeItem(), calculateTotal(), updateStatus(), getDailySales(), getWeeklySales(), getTopSeller()

4.6.3. Specification 3

- **Requirement ID:** CLASS3
- **Description:** Product Class
- **Requirement:** The Product class shall represent all available products or services and handle inventory management.
  Attributes: ProductID, SupplierID , ProductName, Category, UnitPrice, StockQuanity

  Methods: updateStock(), adjustPrice(), getProductInfo()

4.6.4. Specification 4

- **Requirement ID:** CLASS4
- **Description:** Supplier Class
- **Requirement:** The Supplier class shall store and manage supplier information and provide methods for updating details and viewing order history.
  Attributes: SupplierID, SupplierName, SupplierAddress, SupplierCity, SupplierState, SupplierZipCode, SupplierPhone, SupplierContact, SupplierEmail

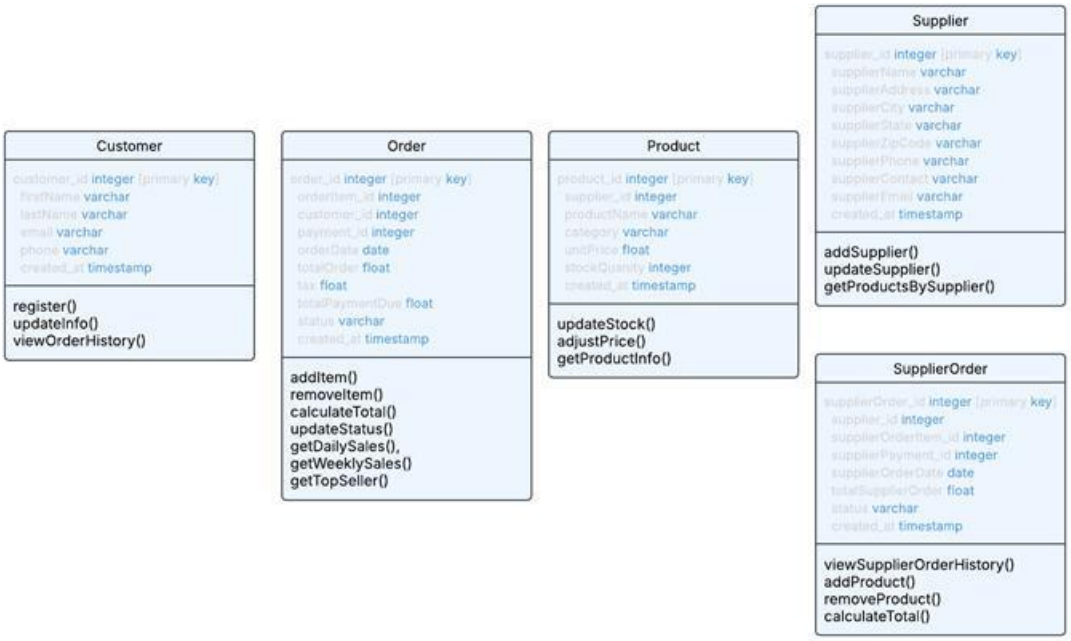
  Methods: addSupplier(), updateSupplier(), getProductsBySupplier()

4.6.5. Specification 5

- **Requirement ID:** CLASS5
- **Description:** Supplier Order Class
- **Requirement:** The Supplier class shall represent individual supplier orders and manage their creation, modification and completion.
  Attributes: SupplierOrderID, SupplierID, SupplierOrderItem, SupplierPaymentID, SupplierOrderDate, TotalSupplierOrder, Status

Methods: viewSupplierOrderHistory(), addProduct(), removeProduct(), calculateTotal()

4.6.6. Class Diagram



4.7. Data Flow and States (Required)

4.7.1. Specification 1

- **Requirement ID:** CLASS1
- **Description:** Handles the customer's information from the moment of creation and additional updates
- **Requirement:** The system shall allow users to create a new account with additional information and be able to edit or update their account after

4.7.2. Specification 2

Requirement ID: CLASS2
Description: Handles orders that are received and customized
Requirement: The system allows for a user to login, create a new order and customize the order to be sent through to the business

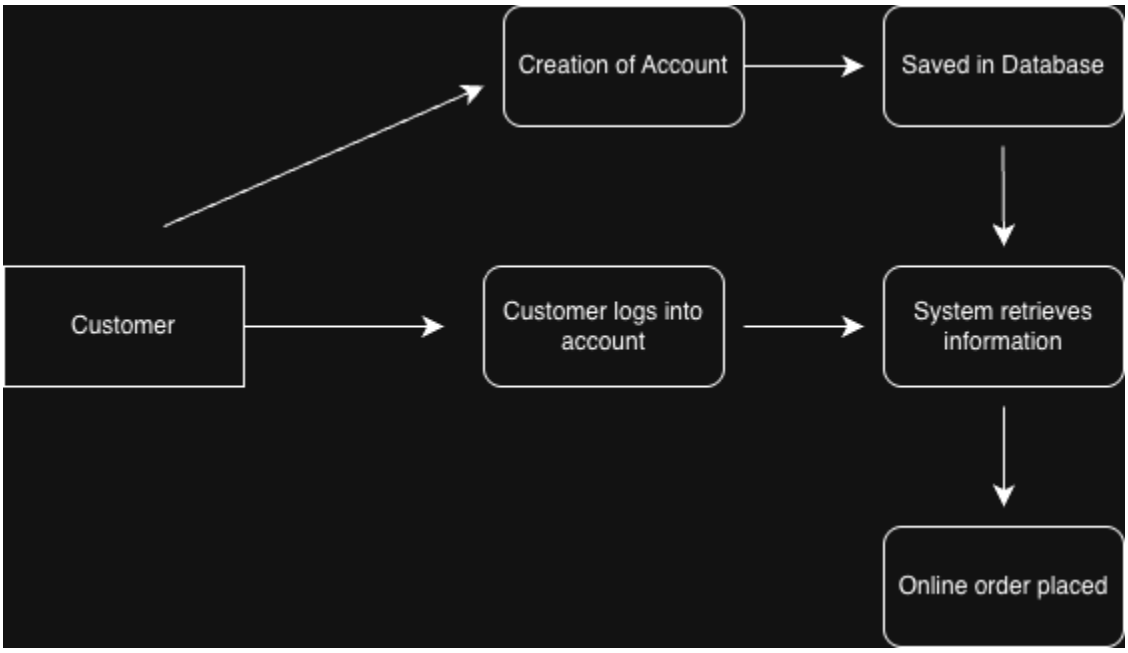
4.7.2. Specification 3

Requirement ID: CLASS3

Description: Inventory Management

Requirement: Administrator counts and updates the current inventory which also updates the online inventory for ordering

4.7.3. Data Flow Diagram (DFD) - CLASS1 example



4.8. Reports (Required)

The Orderly system will include built-in reporting tools designed to help businesses track performance, identify trends, and support informed decision-making. Reports will be generated dynamically from the database and displayed in both tabular and graphical formats within the administrative dashboard. Authorized users will be able to filter reports by date range, product category, and order status.

4.8.1. Specification 1

Requirement ID: REPORT1

Description: Sales Performance Report

Requirement: The system shall generate daily, weekly, and monthly sales reports summarizing total transactions, gross revenue, taxes collected, and top-selling products.

Details:

- Reports will include data visualizations (e.g., bar charts or line graphs) for trends over time.
- The report can be exported as a PDF or CSV file for business records.
- Metrics include total orders, average order value, and sales by category.

4.8.2. Specification 2

Requirement ID: REPORT2
Description: Inventory Status and Restock Report
Requirement: The system shall generate an inventory report identifying current stock levels, low-stock items, and restock recommendations based on historical sales data.
Details:

- Low-stock thresholds will trigger automatic notifications for administrators.
- The report can be viewed in real time or scheduled for automatic weekly delivery.
- Data will include product ID, name, category, quantity remaining, and supplier information.

4.8.3. Specification 3

Requirement ID: REPORT3
Description: Customer Order History Report
Requirement: The system shall provide customer-specific order history reports showing purchase frequency, total spending, and most frequently ordered items.
Details:

- Accessible only by business administrators or the customer associated with the account.
- Used to analyze purchasing behavior and inform marketing or restocking strategies.
- Report includes date of order, order ID, total cost, and items purchased.

4.9. Internal Interfaces (Optional)

Requirement ID: INT1
Description: Customer to Order Interface

The interface between the customer database and order database should exchange between each other so that customers may look at past orders, re-order, customize, or create new orders

Requirement ID: INT2
Description: Orders to Product Interface

The interface between orders database and product database should provide real updates and specifications for products and if there are customizations and additional products to add-on

Requirement ID: INT3
Description: Supplier to Product Interface

The interface between supplier database and product database should be able to easily manage products between a supplier and current inventory, which allows for an easier order system relationship to the supplier

4.10. External Interfaces (Optional)

Requirement ID: EXT1
Description: Payment Gateway Integration (Future Enhancement)
Requirement:
The system could integrate with third-party payment services such as **PayPal** or **Stripe** through secure RESTful APIs. These integrations would allow real-time payment processing, transaction tracking, and automated receipts for customers and businesses.

Specification 1:
• For the prototype, payment transactions will be **simulated** for demonstration purposes.
• Future versions will include **real API connections** to process secure payments using token-based authentication.
• All communication with external payment gateways will occur over **HTTPS** with **SSL encryption** for data protection.

Requirement ID: EXT2
Description: Email Notification Service
Requirement:
The system shall integrate with an **external email service** such as **SendGrid** or **Gmail API** to send automated messages, including order confirmations, updates, and promotional offers.

Specification 2:
• The backend will trigger automated email notifications after successful order placement or when order status changes.
• Messages will include relevant details (order ID, items, total cost, estimated delivery).
• Emails will use standardized templates to maintain branding and readability.

Requirement ID: EXT3
Description: Supplier System Integration (Future Enhancement)
Requirement:
The system could connect to **external supplier APIs** or vendor platforms to automate inventory restocking and supply chain communication.

Specification 3:
• When stock levels fall below a threshold, the backend may generate a **supplier order request** through API calls.
• Business administrators can review and approve these requests before they are submitted to suppliers.
• This integration will streamline supply chain management and reduce manual ordering.

4.11. Other Outputs (Optional)

< List possible outputs generated by the system, such as: Printouts, Web pages, Data or image files, Audio/video outputs, Automated emails or text notifications.>

4.12. Configuration Data (Optional)

< Specify which data or parameters can be configured by users or administrators. Examples include: system settings, default values, access permissions, or file paths**>**

4.13. Training (Optional)

< Describe the training plan to support end users. Specify the format(s) that will be provided, such as: In-person or online sessions, Printed materials, Instructional videos,
Interactive tutorials, Online documentation or help center>

5. Appendixes (Optional)